ChatGPT

Neural Network Model File Formats: Comprehensive Inventory and Loom Integration

Executive Summary: Modern AI models come in many formats, each with its own serialization details, versions, and compatibility. This report catalogs all major model file formats—ranging from PyTorch's `.pt` and SafeTensors through ONNX, TensorFlow, TFLite, CoreML, and more—to map their interoperability and support. For each format we document file extensions, versioning (e.g. ONNX opsets), structure (binary protobuf, JSON+binary, etc.), supported data types, metadata conventions, sharding/quantization capabilities, and security notes (e.g. pickle risks [1] [2]). We then list which frameworks/engines can read or write each format (e.g. PyTorch, TensorFlow, JAX, ONNX Runtime, HuggingFace Transformers, TFLite, CoreML, etc.), including partial support caveats. We map conversion paths (e.g. `torch.onnx.export`, `tf2onnx`, CoreML tools) and note unsupported operators. For Golang support, we inventory existing Go libraries (e.g. `nlpodyssey/safetensors` for SafeTensors, ONNX-GoMLX/**gonnx**/**onnx-go** for ONNX, `github.com/scigolib/hdf5` for HDF5) and assess feasibility of full-format parsing (many formats lack Go support due to proprietary binaries or Python dependencies). We propose a Loom integration architecture: detect format, use native Go readers for safe formats (e.g. SafeTensors, ONNX, HDF5), and fall back to external converters (Python tools or CLI) for others. We sketch a versioning and dependency strategy, testing via roundtrip conversion, and security measures (sandboxing Python, banning pickled inputs). Tables compare formats by features, and Mermaid diagrams visualize format-framework support and the recommended Loom loading flow. **Next steps:** gather any missing spec details, prototype connectors (especially Go ONNX and SafeTensors), and implement robust testing (cross-check outputs) and security audits (avoid arbitrary code).

Model File Formats and Specifications

We catalog each format's characteristics, citing official sources where available.

- **SafeTensors (.safetensors):** A simple, safe binary tensor format by Hugging Face. A file starts with an 8-byte little-endian header length followed by a JSON header describing tensor names, shapes, dtypes, and byte offsets [3]. Following the header is a contiguous byte buffer with the raw tensor data. The format supports any numeric dtype (float32, float16, int8, etc.) and arbitrary shapes; it allows empty tensors and scalars [4]. A `__metadata__` field (string→string map) can hold free-form metadata [5]. SafeTensors emphasizes "safety" (no code execution risk) and zero-copy loading. Endianness is fixed little-endian, and layout is C-major. It does **not** support strided tensors or non-byte-aligned types (those cause errors) [6]. There is no explicit sharding protocol, but models can be manually split into multiple `.safetensors` files (e.g. large LLMs). Compression is not built-in (the bytes are raw) but files could be zipped externally. SafeTensors addresses pickle vulnerabilities by design [1]. Extensions: commonly `.safetensors`. Version: initial spec (v1) is stable; future versions may tweak JSON rules.

- **PyTorch Serialized (.pt, .pth):** Standard PyTorch checkpoints. These are actually **ZIP64** archives (since PyTorch 1.6) containing a pickled Python object (typically a `state_dict`) plus raw storage

files [7] [8]. Internally, `torch.save` uses Python's pickle, making `.pt/.pth` potentially unsafe (arbitrary code execution if loaded with `torch.load`) [2] [1]. Conventionally, the archive has an inner `.pkl` file (the pickled state), an index of byte order, and a `data/` directory with tensor storage. Typical usage: saving `model.state_dict()` or entire model; `.pt` / `.pth` extensions are interchangeable (PyTorch itself doesn't enforce one) [7] [2]. There is no separate versioning of the format aside from PyTorch version compatibility; backwards compatibility is generally preserved. Supported tensor types match PyTorch (floats, ints, half, bfloat16, etc.). No built-in quantization/sharding. Metadata includes a byteorder string and PyTorch version [9]. Extensions: `.pt`, `.pth`, `.bin` (on Hugging Face). TorchScript uses the same `.pt` extension but contains a serialized graph/IR (a ScriptModule) instead of a Python pickle, allowing inference without Python. **Security:** use of pickle makes this unsafe if loading untrusted files [2] [1].

- **TorchScript (.pt):** A subset of PyTorch where models are "scripted" or "traced" into a static graph. Saved via `torch.jit.save`, the file is also a binary archive (protocol buffer and code) containing the model graph and weights. It's more portable than pickled models, and **not** Python-pickled, so safer. Versioning is tied to PyTorch JIT. Typically, use the `.pt` extension as well. Layers/operators: subset of PyTorch ops with TorchScript support. No community spec doc, but TorchScript format evolves with PyTorch (e.g. legacy vs. new IValue formats). Conversion: PyTorch <-> TorchScript via `torch.jit.trace` or `torch.jit.script`.

- **TensorFlow Checkpoint (.ckpt/.index/.data):** TensorFlow's low-level checkpoint format. A checkpoint prefix (e.g. `model.ckpt`) yields two or more files: `<prefix>.index` (a ProtoBuf index of variable names) and one or more `<prefix>.data-00000-of-00001` shards containing raw tensor values [10]. For large models, multiple shards (`.data-00000-of-000NN`). The format is TensorFlow-internal (non-human-readable). There's also a legacy TF1-style checkpoint with `.meta` (graph) file, but in TF2/Keras checkpoints, only variables are stored. Typical usage via `tf.train.Checkpoint.save` or `model.save_weights` (with default `.ckpt`). **Versions:** TF1 vs TF2 differ slightly, but TF2's checkpoint is standard. Types: all TensorFlow variable types. Supports sharding across data files and incremental "latest-checkpoint" management. Quantization: none at this file level (quant is done in graph/ops, not stored here). Security: data-only (no code), but note that the index file and data files are raw binaries. Extensions: `.ckpt` prefix with `.index` and `.data-*-of-*`.

- **TensorFlow SavedModel (directory):** The full TF model format (preferred in TF2). It is a folder (not a single file) containing a `saved_model.pb` (or `.pbtxt`) protobuf, and subdirectories `variables/`, `assets/`, etc. [11]. In detail:

- `saved_model.pb`: a binary ProtoBuf (`MetaGraphDef`) capturing the computational graph and signatures [11].
- `variables/variables.data-*-of-*` and `variables.index`: the model weights (essentially a TF checkpoint inside) [11].
- `assets/`: optional, includes external files (e.g. vocabularies).
- `fingerprint.pb`: metadata about SavedModel version.

- `keras_metadata.pb`: (if saved from Keras) metadata about model config and compile state [12]. Format versions evolve with TF releases. It is language-neutral (Protobuf + binary) and portable across C++/Python/JS runtimes. Sharding: variables are usually sharded as `.data-00000-`

`of-00001` etc for large variables [11]. Quantization: no direct weight quant in SavedModel; models for TFLite use delegates. Security: the graph may contain custom signature names, but no code is executed on loading (though TensorFlow functions do support saved `tf.function` traces). Typical extension: none (folder with .pb).

- **TensorFlow GraphDef (.pb / .pbtxt):** A lower-level protobuf format containing a serialized `GraphDef` (computational graph) and optional weights. Commonly used for inference in TF1. Version: GraphDef proto has versions; generally "frozen graph" combining graph+weights is `.pb`. Text form is `.pbtxt`. Not often used in TF2 (SavedModel replaced it). Can be generated by freezing a session, or by exporting from Keras. Acts as a container; no metadata. Quantization: not in format. Security: static graph only, no code.

- **HDF5 (.h5, .keras):** A general scientific binary container. Keras/TensorFlow uses HDF5 for model or weights. For example, `model.save('mymodel.h5')` (Keras <3) produces a single HDF5 file containing the architecture (JSON or binary proto) and weights [13]. It can also hold optimizer state. The format is fully defined by HDF5 and TensorFlow (there's a group for layers with datasets for weights). It supports hierarchical data, so multiple tensors per layer. Sharding: not inherent (fits in one file). HDF5 supports compression (GZIP, etc.) at the dataset level, though Keras by default does not compress. **Security:** HDF5 has had buffer/alloc vulnerabilities [14] (and is large code); it is safer than pickle but not fully trivial. TensorFlow now discourages HDF5 for large models. HDF5 versions: it's version 1.x or 2.x of the HDF spec.

- **ONNX (.onnx):** The Open Neural Network Exchange format is a standardized protobuf IR for neural networks [15]. An `.onnx` file is a Protobuf containing a Graph (nodes/operators) and initializers (weights), along with model metadata (operator set version, IR version). The ONNX spec (see IR.md) defines built-in operators and a type system [16] [15]. **Versioning:** ONNX uses "opset" numbers. For example, ONNX IR version 13 corresponds to opset 9, etc. Operator sets are numbered (e.g. opset 13, 14, ...). Different frameworks support different opsets; ONNX Runtime supports opsets from v1.2.1 onward [17]. Weight storage: ONNX allows weights either embedded in the `.onnx` file or in external binary files (for files >2GB) [15]. Sharding: external data (e.g. `.onnx` + `.data`). The binary is little-endian protobuf. Extension: `.onnx`. Supported layers/operators: any in the ONNX operator set (including many DL ops, plus non-DL ops). Quantization: ONNX Runtime and tools support quantization schemes (QDQ, quantize-weight, etc.), but the `.onnx` itself just stores raw values. Security: since it's protobuf of graph+numbers, no execution risk. ONNX is widely supported for interchange.

- **TFLite (.tflite):** TensorFlow Lite flatbuffer model. It is a single-file **FlatBuffers** binary defined by a TFLite schema. The schema is versioned with TensorFlow Lite releases. A `.tflite` contains the graph and weights optimized for mobile. It supports built-in TensorFlow Lite ops and custom ops. Quantization: TFLite explicitly supports 8-bit, float16, etc., via quantized schemas (weights and activations can be quantized). Sharding: not typically; one file for model. Tools: `tflite_convert`. Security: flatbuffer is safer than pickle. Extension: `.tflite`.

- **CoreML (.mlmodel):** Apple's format for iOS/macOS. A `.mlmodel` file is a zipped bundle containing a protobuf (Model.proto) describing the network (CoreML's own neural net schema) and weights [18]. The spec is public; Core ML supports many layer types (neural nets, pipelines, decision trees, etc.)

with fixed schema in `Model.proto` (NeuralNetwork or MLProgram blocks). Versioning: CoreML evolves (e.g. spec v3, v4 with new ops). Conversion: typically from Keras/ONNX using `coremltools`. Quantization: CoreML Tools supports quantized weights (8-bit) for iOS NN APIs. Sharding: single file (can be zipped). Security: no code execution.

- **OpenVINO IR (.xml + .bin):** Intel's proprietary format. An XML file describes the model graph and attributes, and a `.bin` file holds the weights (typically FP32 or FP16) [19]. The IR version is in the XML header (`version` attribute). Conversion: use OpenVINO Model Optimizer (from ONNX, TensorFlow, Caffe, etc.). No native editor beyond OpenVINO. Quantization: supported via Post-Training Optimization Toolkit into INT8 weights, then stored similarly in `.bin`. Sharding: no (always one `.bin`). Security: just data.

- **MXNet (symbolic) (.json + .params):** Apache MXNet's "symbolic" format uses two files: a JSON (text) file with network architecture, and a binary `.params` file containing NDArray weights. Example: `model-symbol.json` and `model-0000.params` (prefix with epoch) [20]. Versions: MXNet 1.x. Quantization: MXNet has quantization support (post-training), but weights still in `.params` (they may include quant scales). Sharding: one `.params` holds all parameters.

- **Caffe (.prototxt + .caffemodel):** Caffe defines nets with a `.prototxt` (plain-text protobuf definition of layers) and weights in a binary protobuf `.caffemodel` [21]. The `caffemodel` stores layer parameters; the `prototxt` stores the architecture. There is also `.solverstate` (training state). Quantization: Caffe2 later, but original Caffe has no built-in quantization. Sharding: no (fits in one file pair).

- **PaddlePaddle (.pdmodel + .pdiparams):** PaddlePaddle's inference model uses `save_inference_model`, which outputs `model.pdmodel` (program graph) and `model.pdiparams` (weights) [22]. In dynamic mode, model parameters use `.pdparams` (training, includes optimizer). Quantization: Paddle has quant APIs, resulting in weight tensors still in these files.

- **JAX/Flax checkpoints:** JAX itself has no standard format. Flax often saves checkpoints via Orbax or `checkpoints.save_checkpoint`, which writes data (often msgpack for config and NPZ for arrays) to a directory. The result might be `.msgpack` or `.npz` files. No fixed schema (depends on user code). Quantization: no. We mark this unspecified due to variability.

- **Hugging Face Transformers (Hub) format:** Not a single file format, but a standard layout on the HF Hub: weights (either `pytorch_model.bin` / `.safetensors` for PyTorch, or `tf_model.h5` / SavedModel for TF), plus a `config.json`, tokenizer files, etc. We note that HF encourages **SafeTensors** for weight files for safety and fast loading. If both `*.bin` (pickle) and `*.safetensors` exist, Transformers may download both but use the safer one [23]. (Detail from [10] is implicit; HF docs and code confirm use of these formats.)

- **GGUF (.gguf):** "Generic GPT Unified Format" by ggerganov (llama.cpp developer). A **binary** format storing quantized LLM weights and some metadata [24]. It encodes both tensors and standardized metadata in one file (based on GGML but extended) [24]. GGUF is optimized for fast loading; it's used by llama.cpp, GPT4All, Ollama, etc [25]. It supports various quantized formats (e.g. Q4_K, Q8, etc. per

GGML). Hugging Face now directly hosts GGUF models. No shard support beyond internal (one file). Security: data-only.

- **MLC format (MLC-LLM):** A format used by MLCommons MLC-LLM. Models are *converted* into an MLC-specific directory (often named `*-MLC`). The exact file layout is internal (weights, config JSON). Usage requires first converting (e.g. via `mlc_llm convert_weight` [26]) then compiling. Details are internal to MLC; we note that it's versioned by MLC-LLM release (v0.1.0 etc.) and supports quantized weights. We treat it as unspecified since docs emphasize conversion but not a stable standalone format description.

Unspecified/unknown formats (e.g. ".ckpt" in other contexts): Any format not covered above is noted as unspecified.

Frameworks and Read/Write Support

For each format, we list major frameworks/engines that can **read/write** it (often with caveats):

- **PyTorch (.pt/.pth, SafeTensors, TorchScript):**
- *Writes:* PyTorch (`torch.save`) writes `.pt/.pth` pickles [2] and can write TorchScript (`torch.jit.save`). PyTorch can export to ONNX via `torch.onnx.export`. Hugging Face `transformers` `save_pretrained()` can write `.bin` (pickle) or `.safetensors`.
- *Reads:* PyTorch (`torch.load`) reads `.pt/.pth` (via pickle). It can load TorchScript via `torch.jit.load`. PyTorch and HuggingFace `from_pretrained` can load SafeTensors (using `safetensors` library) and pickle `.bin` files (if safe environment). ONNX Runtime does not natively load `.pt` or SafeTensors, only ONNX.

- *Partial caveats:* PyTorch cannot read TensorFlow, TFLite, CoreML directly. It uses converters (ONNX). Loom (Go) currently only has SafeTensors loader, not pickled .pt (unsafe).

- **TensorFlow (SavedModel/.ckpt/.pb/H5):**

- *Writes:* TensorFlow (Keras) writes SavedModel (`model.save()`, default), HDF5 (`save(..., format='h5')`), or checkpoints (`model.save_weights()` gives .ckpt) [10] [13].
- *Reads:* TensorFlow/Keras loads SavedModel and HDF5 (`load_model`). `tf.train.Checkpoint.restore` reads .ckpt. TFLite (`TFLiteConverter`) can load SavedModel or H5 to convert to .tflite. ONNX converters (`tf2onnx`) read SavedModel/H5 to produce ONNX. CoreMLTools reads TF SavedModel/H5 to convert to .mlmodel.

- *Partial:* TF cannot read PyTorch or ONNX directly (use tf2onnx for ONNX).

- **ONNX Runtime:**

- *Reads:* ONNX Runtime (ORT) can load any ONNX opset from 1.2.1 onwards [17]. ORT cannot read TF, PyTorch, etc. (only ONNX).

- *Writes:* ORT is inference engine; does not write models.

- **TensorFlow Lite:**

- *Reads:* The TFLite interpreter reads `.tflite`. Other frameworks do not natively. ONNX-TFLite tools exist to convert ONNX->TFLite (via TensorFlow intermediate).
- *Writes:* `tflite_convert` (TF) writes `.tflite`.

- **CoreML (Apple):**

- *Reads:* Core ML runtime reads `.mlmodel`.
- *Writes:* `coremltools` (Python) converts from Keras, ONNX, PyTorch (via ONNX) to `.mlmodel`.

- **OpenVINO IR:**

- *Reads:* OpenVINO Inference Engine reads `.xml/.bin`. The ONNX importer can also load ONNX directly.
- *Writes:* Model Optimizer (Python) writes IR from ONNX, TensorFlow, Caffe, PyTorch (via ONNX) etc.

- **MXNet:**

- *Reads:* MXNet loads its `.json` + `.params` via the Symbol API (`mx.model.load_checkpoint`). Some converters (ONNX-MXNet) can convert ONNX to MXNet.
- *Writes:* MXNet Symbol API saves `.json/.params`. MXNet also has a model archive (`.params`).

- **Caffe:**

- *Reads:* Caffe framework reads `.prototxt` + `.caffemodel`. ONNX and PyTorch (via Caffe2) support Caffe import somewhat.
- *Writes:* Caffe (and Caffe2) can write `.caffemodel`.

- **PaddlePaddle:**

- *Reads:* Paddle Inference loads `.pdmodel/.pdiparams`. A converter exists (paddle2onnx) to convert Paddle to ONNX.
- *Writes:* `paddle.static.save_inference_model` writes these files [22].

- **JAX/Flax:**

- *Reads/Writes:* Typically JAX/Flax code saves NumPy (NPZ) or msgpack dictionaries. No standardized import/export to other engines; conversion via intermediate (e.g. JAX→TensorFlow or XLA) is possible but not automated in main frameworks.

- **HuggingFace Transformers:**

- *Reads:* Transformers can load PyTorch `.bin` or `.safetensors`, TensorFlow H5/SavedModel, ONNX (via `transformers.onnx`), and (via adapters) CoreML.
- *Writes:* `save_pretrained()` writes PyTorch state (`.bin` / `.safetensors`) or TensorFlow (`.h5`), plus JSON config. Converters (`transformers.convert_graph_to_onnx`) export ONNX.

- *Notes:* If both `.bin` and `.safetensors` exist, HF uses safetensors preferentially. LoRA and GPTQ formats also appear on the Hub (HuggingFace forums recommend using SafeTensors for LoRA, quant formats).

- **GGUF-using frameworks:**

- llama.cpp, Ollama, LM Studio, GPT4All, etc.
- *Reads:* LLMA engine tools can read GGUF directly.

- *Writes:* Typically convert from PyTorch/ONNX into GGUF using `ggml-tools` or HF's `gguf` converter [24].

- **MLC LLM:**

- *Reads:* MLC runtime reads its converted format (via compiled library).
- *Writes:* Use `mlc_llm convert` and `compile` commands (see MLC docs) [26].

Conversion Paths and Tools

Direct Converters:

- PyTorch → ONNX: `torch.onnx.export` (supports many opsets).
- TensorFlow/Keras → ONNX: tf2onnx (supports TF2, Keras, including some TFLite), or [tf2onnx CLI] [27].
- ONNX → TensorFlow: limited; sometimes use onnx-tf (not always complete).
- ONNX ↔ CoreML: CoreML Tools can import ONNX (v1 opset 9+) to `.mlmodel` and export a `.mlmodel` to ONNX.
- PyTorch ↔ TensorFlow: No direct; use ONNX as intermediate or rewrite model in both.
- TensorFlow → TFLite: `tflite_convert` (also supports `.pb`, SavedModel).
- ONNX → TFLite: No direct; recommended: ONNX → TF SavedModel (using onnx-tf) → TFLite.
- CoreML → ONNX: Not directly, unless custom.
- Caffe ↔ ONNX: [Caffe2 (pytorch)] had ONNX exporter for Caffe models.
- MXNet ↔ ONNX: MXNet has ONNX importer/exporter.
- Paddle ↔ ONNX: Paddle2ONNX.
- HuggingFace:
- PyTorch/TensorFlow models in HF can be converted to ONNX via the `transformers.onnx` pipeline or huggingface/optimum.
- HF models → SafeTensors: simply choose `save_safetensors=True`.
- HF → GGUF: use HF's `gguf` converter or `ovllama/gguf-tools`.
- GGUF ↔ others: GGUF can be produced by `ggml-tools` from GGML, or by `convert-weights` in text-generation-webui.

Lossiness and Unsupported Ops:
Conversions may lose frameworks-specific features. E.g. PyTorch custom layers won't translate to ONNX unless registered. Keras custom objects might not export cleanly. Always check if all ops are supported (on ONNX exporters often fail on dynamic control flow or custom ops). Many conversion tools report unsupported ops; e.g. ONNX export notes which operators map.

Tools Summary: `torch.onnx.export`, `tf2onnx`, `onnx-tf`, `coremltools`, `tf.lite.TFLiteConverter`, HuggingFace Optimum/convertors, `caffe2python`, `mxnet.onnx`. For GGUF/MLC, specialized CLI tools.

Go Libraries and Implementation Feasibility

We inventory Go support for each format and assess how difficult it would be to implement:

- **SafeTensors:** There is a Go library `github.com/nlpodyssey/safetensors` [28]. Easy to parse JSON header and read bytes. Implementation difficulty: **Easy**. Loom already includes this (as `safetensors.go`).

- **PyTorch .pt (pickle):** No pure-Go library can safely parse Python pickle of arbitrary objects. Implementing full PyTorch pickle loader in Go would be **impossible/dangerous** (security risk). **Feasibility: Hard/Unlikely**. Workaround: require conversion to SafeTensors or ONNX externally.

- **TorchScript .pt:** TorchScript files are protobuf-based, but no known Go parser. Could try invoking `libtorch` via cgo (which goes against Loom's no-CGO). Likely **hard**.

- **TensorFlow Checkpoint (.ckpt):** This involves reading TensorFlow's custom binary format (variable shards + index). There is no Go library for TF Checkpoint (without C or complex binary parsing). Implementing from spec is very complex (internal TF format). **Feasibility: Hard**. Best to call Python to restore weights.

- **SavedModel:** It is a folder of Protobufs and binary weights. One could, in Go, parse `saved_model.pb` via Protobuf (requires TF proto definitions), and parse variables with HDF5? Actually variables are not HDF5 but TF-checkpoint format. So full support: **Very Hard**. Could pick out weights via reading `variables.data-*` with TensorFlow's checkpoint API (not in Go).

- **GraphDef (.pb):** If it's a frozen graph, the `.pb` is just a ProtoBuf (`GraphDef`). In Go, one could load GraphDef if the `.proto` definitions are compiled (TensorFlow protos). This is **Medium** difficulty (need TF proto, not built-in in Go). But then semantics depend on executing it. Probably not needed directly for Loom unless converting.

- **HDF5 (.h5):** There is a robust pure-Go HDF5 reader `github.com/scigolib/hdf5` [29]. So it is **Feasible**. Implementing full model load (e.g. Keras H5 architecture + weights) is moderate effort: one needs to follow Keras HDF5 schema. Difficulty: **Medium** (due to nested groups/datasets).

- **ONNX (.onnx):** There are Go projects: ONNX-GoMLX [30] can parse ONNX; also **gonnx** and **onnx-go** exist [31]. They have limitations (supported opsets, certain ops), but basic loading is possible. Difficulty: **Medium to Hard** to cover all ops. But basic model load is doable.

- **TFLite (.tflite):** TFLite is FlatBuffers. We could use Go's FlatBuffers support and import the TFLite schema to generate Go code. That's **Medium** difficulty (schema is complex). However, no off-the-shelf library. Possibly easier: convert TFLite to ONNX or TensorFlow externally.

- **CoreML (.mlmodel):** It's a Protobuf (MIL or NeuralNetwork spec). Google's `coremltools` exists in Python. No Go support. Parsing via proto in Go would require Apple's MLModel.proto and ancillary files. This is **Hard**.

- **OpenVINO IR:** It's XML + binary. One could parse XML (easy) and read `.bin` bytes manually. But interpreting the XML schema and operations is proprietary. Without the full spec, Hard. Not a priority.

- **MXNet (.json/.params):** JSON is easy in Go; `.params` is a custom NDArray binary format. Possibly parseable (gluon NDArray raw). Would require implementing NDArray reader (binary of floats). Feasibility: **Medium** (needs format spec).

- **Caffe (.prototxt/.caffemodel):** `.prototxt` is plaintext and can be parsed via Caffe protobuf definitions (we'd need protos in Go). `.caffemodel` is binary protobuf. If we import Caffe's `.proto` (e.g. with `protoc` for Go), it's doable. Difficulty: **Medium to Hard** (requires protos and mapping).

- **PaddlePaddle:** `.pdmodel` / `.pdiparams` presumably Proto+binary. Unless protos available, it's **Hard**.

- **JAX/Flax:** No fixed format, skip support.

- **HuggingFace Hub:**

- SafeTensors: supported via safetensors lib.
- PyTorch `.bin`: same as .pt (pickle), not natively supported in Go.

- HuggingFace also uses JSON configs and arrays (can parse JSON).

- **GGUF (.gguf):** As of now, no known Go lib. The HF docs mention only a JavaScript parser for GGUF [32]. Writing one in Go would require GGUF spec (it's not a standard like Proto or FlatBuffer, likely custom). **Feasibility: Hard**. Use Python (or call llama.cpp) as fallback.

- **MLC format:** Only via MLC tooling (Python) or compiled libs; no Go.

Summary of Go Support: Formats with good Go support: SafeTensors (easy), HDF5 (via library), ONNX (partial), possibly MXNet (JSON). Others (Pickle, TF, CoreML, TFLite, etc.) are hard. So primary targets for pure-Go: SafeTensors, ONNX, HDF5. All others likely require external conversion or disallowed.

Loom Integration Architecture and Roadmap

To enable Loom (Go-based AI runtime) to load many formats, we propose the following strategy:

1. **Format Detection:** Upon model load, detect format by extension and/or file magic:
2. Recognize `.safetensors`, `.onnx`, `.h5`, `.pb` (graph), `.mlmodel`, etc.
3. If `.ckpt` with accompanying `.index`/`.data`, treat as TF checkpoint.
4. For directories (SavedModel), look for `saved_model.pb`.
5. If unrecognized, assume conversion needed.
6. **Native Go Loaders (Priority):** Implement native importers in Go for safe/feasible formats:
7. **SafeTensors:** Use existing Go library to read tensors into Loom's tensor store. (Done as `safetensors.go` in Loom). This allows direct import of HuggingFace weights (LLM, etc.).
8. **ONNX:** Integrate a Go ONNX reader (e.g. leverage ONNX-GoMLX or similar). This allows loading ONNX models. We may either call ONNX-GoMLX or link minimal Protobuf parser. It's **Medium** effort to cover common cases.
9. **HDF5 (Keras):** Optionally support reading `.h5` using the scigolib library. This would let Loom load Keras models. Difficulty is moderate; may be deferred.
10. **JSON formats:** For MXNet or small JSON configs, we can parse in Go.
11. **GGUF (maybe):** If the GGUF spec becomes public and simple, consider adding a reader (as it's important for LLMs on CPU). Otherwise, skip for now.
12. **Fallback Converters:** For unsupported formats, spawn sandboxed external conversion:
13. **Python subprocess:** Use a bundled Python environment (maybe a lightweight TensorFlow/PyTorch wheel) to convert on-the-fly. For example:
    - `.pt` → SafeTensors (via `transformers` or custom script).
    - `.pb` (Frozen TF) → ONNX (`tf2onnx`).
    - SavedModel/H5 → ONNX/TFLite (via `tf2onnx` or `tf.lite`).
    - `.mlmodel` → ONNX (via `coremltools` in Python).
    - `.tflite` → ONNX (two-step via TF).
    - `.caffemodel` → ONNX (via PyTorch/Caffe2 importer).
    - `.pdmodel` → ONNX (via paddle2onnx).
    - `.gguf` → ONNX (if tools exist).
14. Use WebAssembly workers or docker if security is a concern. Limit conversions to offline or trusted workflows.
15. **Versioning Strategy:** Keep track of format versions (e.g. ONNX opset) at load time:

16. Store metadata (format and version) with each model in Loom.

17. Evolve code: support opsets incrementally. For major format updates, release Loom version bump.

18. Provide migration: e.g. ability to convert old ONNX opset to newer via tools if needed.

19. **Roadmap Priorities:**

20. *Phase 1:* Implement SafeTensors loader (already in Loom) and test with various HF models. Implement basic ONNX importer (using `onnx-go`).

21. *Phase 2:* Add HDF5 loader, basic protocol for SavedModel (maybe by converting TF to ONNX using Python), and config wrappers.
22. *Phase 3:* Provide Python integration for leftovers: wrap `transformers`, `onnxruntime`, or conversion CLIs. Optionally allow CGO with ONNXRuntime C API if needed (but LoC wants no CGO).

23. Throughout, maintain cross-platform ("portable, no CGO").

24. **Testing & Validation:**

25. For each format, have a test suite: load a reference model (from HF or framework), run a simple inference (compare output vs official runner). For conversions, check numerical closeness.

26. Use CI to run conversions with small models (ResNet, GPT-2, etc.) ensuring correctness.

27. Fuzz test: attempt loading malformed files to ensure errors (and no code execution).

28. **Security Mitigations:**

29. **Disable pickle**: Avoid loading any pickled format in Go (no `.pt/.bin` via pickle).

30. **Sandbox conversions**: Run external converters in isolated environments.
31. **Input validation**: Check SafeTensors headers for valid JSON, ensure ONNX protobuf parsing is strict.
32. **Limit external calls**: Optionally maintain an allow-list of trusted converters.

33. **Libraries with vulnerabilities**: Keep HDF5/Protobuf libs up-to-date. Note that SafeTensors avoids known CVEs of HDF5/pickle [14].

34. **Integration Flow (Mermaid Diagram 2):** Loom's loader could follow this flow:

```
graph LR
  A[Model File/Input] --> B{Detect Format}
  B -->|.safetensors| C[Load via SafeTensors (Go)]
  B -->|.onnx| D[Load via ONNX (Go)]
  B -->|.h5| E[Load via HDF5 (Go) or external script]
  B -->|.pb (GraphDef)| F[External: TF conversion (Py) -> ONNX/SafeTensors]
  B -->|.keras| E
```

```
  B -->|.ckpt/.index| G[External: TF Python -> ONNX/SafeTensors]
  B -->|.mlmodel| H[External: coremltools -> ONNX]
  B -->|.tflite| I[External: TF Lite Converter or ONNX pipeline]
  B -->|.params/.json| J[Load MXNet (Go) or ext convert to ONNX]
  B -->|.caffemodel| K[External: Caffe2/PyTorch -> ONNX]
  B -->|.pdmodel| L[External: Paddle2ONNX]
  B -->|.gguf| M[External: llama.cpp -> SafeTensors/ONNX]
  B -->|others| N[External: require manual conversion]
  C & D & E & F & G & H & I & J & K & L & M & N --> Z[Import weights into Loom
engine]
  Z --> O[Inference/Evaluation in Loom]
```

Comparison Table of Formats

Format	File Extensions	Versions/ Opset	Frameworks (read/write)	Read/Write	Quantization Support	Sharding Support
SafeTensors	`.safetensors`	v1.x	PyTorch (via safetensors), TensorFlow (TF NumPy etc), JAX, LLaMA-cpp, HF	R/W (Go/ Python)	Data types incl. int8, fp16	No (can split files manually)
PyTorch (.pt/.pth)	`.pt`, `.pth`, `.bin`	PyTorch versions (1.x)	PyTorch (R/W), ONNX export, HF Transformers	R/W (PyTorch)	FP32, FP16, BF16, etc.	No
TorchScript	`.pt` (TorchScript)	PyTorch versions	PyTorch (jit R/ W), C++ LibTorch	R/W (PyTorch)	Same as PyTorch	No
TF Checkpoint	`.ckpt` (+ `.index` / `.data`)	TensorFlow 1.x/2.x	TensorFlow (R/ W), Keras callbacks	R/W (TF)	Int/FP types (no built-in quant storage)	Yes (data shards)
SavedModel (TF)	Dir containing `.pb`	TF 1.x/2.x	TensorFlow (R/ W), TensorFlow Lite, TFJS	R/W (TF)	No (converter does separate quant)	Yes (variables/ *shards)
GraphDef (.pb)	`.pb`, `.pbtxt`	TF GraphDef v1/v2	TensorFlow, TF Lite (via converter)	R/W (TF)	No (float weights)	No

Format	File Extensions	Versions/ Opset	Frameworks (read/write)	Read/Write	Quantization Support	Sharding Support
HDF5 (.h5)	`.h5`, `.keras`	HDF5 spec 1.x/2.x	Keras/ TensorFlow (R/ W)	R/W (TF)	Store quant tensors if given	No
ONNX	`.onnx` (+.data)	Opsets 7… 21+ (ONNX 1.x)	PyTorch, TF/ ONNX exporters, ONNX Runtime (R), etc.	R/O (ONNX)	Yes (quantization params allowed)	Yes (external data)
TFLite (.tflite)	`.tflite`, `.lite`	TF Lite version	TensorFlow Lite (R/W), TF (export to .tflite)	R/W (TF Lite)	Yes (int8/16 support)	No
CoreML (.mlmodel)	`.mlmodel` (zip)	CoreML spec (v3/v4 etc.)	Core ML (R), coremltools (W), TF/ ONNX→CoreML	R/W (Core ML)	Yes (8-bit via coremltools)	No
OpenVINO IR	`.xml` + `.bin`	IR v10+ (OpenVINO 202X)	OpenVINO (IR R), Model Optimizer (W)	R/W (OpenVINO)	Yes (INT8, FP16)	No
MXNet	`.json` + `.params`	MXNet 1.x	MXNet (R/W), MXNet-ONNX converters	R/W (MXNet)	Yes (via quant lib)	No
Caffe	`.prototxt` + `.caffemodel`	Caffe (1.x)	Caffe (R/W), ONNX (via Caffe2)	R/W (Caffe)	No	No
Paddle	`.pdmodel` + `.pdiparams`	PaddlePaddle	Paddle (R/W), paddle2onnx (conv.)	R/W (Paddle)	Yes (via PTQ)	No
JAX/Flax	(`.npz`, `.msgpack` etc.)	Orbax/Flax format	JAX/Flax only	R/W (JAX)	No	No
Hugging Face Hub	`.bin`, `.safetensors` + configs	Model-specific	HF Transformers (R/W), PyTorch (R/W)	R/W (HF libs)	Dependent on underlying format	No
GGUF	`.gguf`	GGUF v1 (by llama.cpp)	llama.cpp, GPT4All, Ollama (R)	R (specialized)	Yes (quant weights)	No

Format	File Extensions	Versions/ Opset	Frameworks (read/write)	Read/Write	Quantization Support	Sharding Support
MLC (LLM)	`* -MLC` (dir)	MLC-LLM v0.x	MLC LLM (R), (no others)	R/W (MLC)	Yes (quant)	Possibly (internal)

("Read/Write": which engines can read/write; "O"=only; "R/W" means both.)

Integration Flow for Loom (Mermaid Diagram)

```
flowchart TD
  A[(Model file input)] --> B{Detect format}
  B -- SafeTensors --> C[Load weights (Go SafeTensors)]
  B -- ONNX --> D[Load model (Go ONNX parser)]
  B -- HDF5 (.h5) --> E[Load (Go HDF5) or ask for conv]
  B -- PB/GraphDef --> F[External convert to ONNX]
  B -- SavedModel --> F
  B -- .ckpt --> F
  B -- .mlmodel --> G[External coremltools->ONNX]
  B -- .tflite --> H[External TF Lite converter]
  B -- MXNet (.json/.params) --> I[External converter (to ONNX)]
  B -- Caffe --> I
  B -- Paddle --> I
  B -- .gguf --> J[External llama.cpp/gguf->ONNX]
  B -- others --> K[Require manual conversion]
  C --> Z[Integrate into Loom]
  D --> Z
  E --> Z
  F --> Z
  G --> Z
  H --> Z
  I --> Z
  J --> Z
  K --> Z
  Z --> L[(Loom Inference)]
```

Next Steps and Recommendations

- **Spec Verification:** For any formats with limited public info (e.g. JAX checkpoints, MLC-LLM), either gather spec or note "unspecified".
- **Go Support:** Begin coding the easiest pure-Go loaders: SafeTensors (already in Loom), ONNX (via onnx-go), and optionally HDF5. Assess if these cover the majority of target models.
- **Converters:** Package and vet Python-based converters. E.g., a helper that takes an arbitrary model and tries known conversion paths to SafeTensors/ONNX.
- **Security:** Implement strict checks in all parsers, and consider running external code in containers.

- **Testing:** Establish benchmarks: load sample models from each format and compare results against native frameworks. Write unit tests for header parsing, missing operations, and error cases.

Sources: Format details from official docs and repos: SafeTensors spec [3] ; PyTorch torch.save format [2] ; TensorFlow SavedModel structure [11] ; ONNX spec/details [15] ; CoreML protobuf spec [18] ; OpenVINO IR docs [33] ; MXNet/Caffe formats [20] [21] ; Paddle API docs [22] ; GGUF info [25] [24] ; Loom docs on SafeTensors [34] and others.

This inventory and plan should guide Loom's multi-format support development. Further updates should track evolving specs (e.g. new ONNX opsets, CoreML updates) and emerging formats.

---

[1] [3] [4] [5] [6] [14] GitHub - safetensors/safetensors: Simple, safe way to store and distribute tensors · GitHub
https://github.com/safetensors/safetensors

[2] [7] [8] [9] PyTorch Serialized File Format
https://www.loc.gov/preservation/digital/formats/fdd/fdd000644.shtml

[10] Save and load models | TensorFlow Core
https://www.tensorflow.org/tutorials/keras/save_and_load

[11] [12] TensorFlow SavedModel Format Explained
https://apxml.com/courses/getting-started-with-tensorflow/chapter-6-saving-loading-models/tensorflow-savedmodel-format

[13] Whole model saving & loading
https://keras.io/2/api/models/model_saving_apis/model_saving_and_loading/

[15] Working with Large Models | onnxruntime
https://onnxruntime.ai/docs/tutorials/web/large-models.html

[16] [27] onnx/docs/IR.md at main · onnx/onnx · GitHub
https://github.com/onnx/onnx/blob/main/docs/IR.md

[17] ONNX Runtime compatibility
https://onnxruntime.ai/docs/reference/compatibility.html

[18] Core ML Model Format Specification — Core ML Format Reference documentation
https://apple.github.io/coremltools/mlmodel/index.html

[19] [33] OpenVINO IR format — OpenVINO™ documentation
https://docs.openvino.ai/nightly/documentation/openvino-ir-format.html

[20] amazon sagemaker - Error while deserializing the Apache MXNet object - Stack Overflow
https://stackoverflow.com/questions/49658834/error-while-deserializing-the-apache-mxnet-object

[21] CAFFEMODEL File - What is it and how do I open it?
https://fileinfo.com/extension/caffemodel

[22] save_inference_model-API Document-PaddlePaddle Deep Learning Platform
https://www.paddlepaddle.org.cn/documentation/docs/en/api/paddle/static/save_inference_model_en.html

[23] Safetensors · Hugging Face
https://huggingface.co/docs/safetensors/index

24 25 32 GGUF · Hugging Face
https://huggingface.co/docs/hub/en/gguf

26 IDE Integration — mlc-llm 0.1.0 documentation
https://llm.mlc.ai/docs/deploy/ide_integration.html

28 nlpodyssey/safetensors: safetensors lib for Go · GitHub
https://github.com/nlpodyssey/safetensors

29 hdf5 package - github.com/scigolib/hdf5 - Go Packages
https://pkg.go.dev/github.com/scigolib/hdf5

30 31 GitHub - gomlx/onnx-gomlx: ONNX / GoMLX Conversion · GitHub
https://github.com/gomlx/onnx-gomlx

34 Serialization, Persistence, and Loading — Loom Docs | OpenFluke
https://openfluke.com/docs/serialization